Codex + T3 Code UI/UX Research Brief for eCode

Reference analysis, visual examples, and a redesign prompt for the next eCode refinement pass

Prepared on 2026-03-16

This brief compares Codex and T3 Code as interface references for eCode, captures the strongest reusable patterns, and packages them into a concrete redesign prompt for an AI UI designer.

Executive Summary

- Codex contributes the broader product posture: the app should feel like a control room for work, not a styling pass on a chat window.
- T3 Code contributes the tighter shell behavior: dark chrome, strong left-rail identity, compact status surfaces, and a bottom composer that remains part of the main conversation flow.
- For eCode, the best hybrid is a dark command-center shell with softer internal cards and clearer action states. Keep the shell dark and calm, but make key content surfaces easier to scan and trust.
- The redesign target is not a pixel clone. Preserve eCode's native Rust and local-first strengths while making hierarchy, density, and flow feel closer to the best parts of both references.

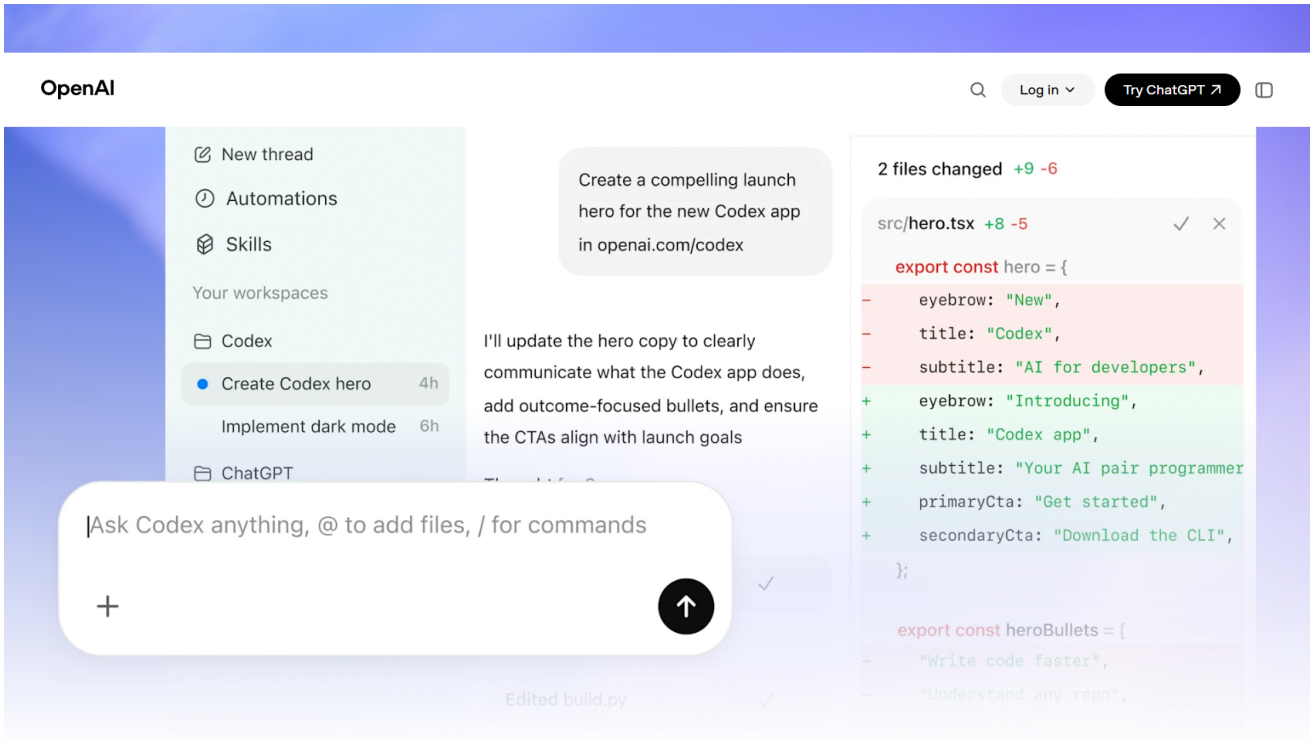
Current eCode Surfaces To Refine

- Top bar: brand, project picker, shell toggles, runtime status, settings entry.
- Left sidebar: thread list and filtering, currently useful but visually lighter than the references.
- Center panel: thread header, transcript, approvals, and the main composer.
- Right panel: plan and diff context, currently baseline rather than a fully integrated command center.
- Settings modal: functional, but still reads as a separate utility screen instead of part of the same product language.
- Status bar and terminal: valuable for trust and local execution, but their visual hierarchy can be tighter.

Codex UI/UX Findings

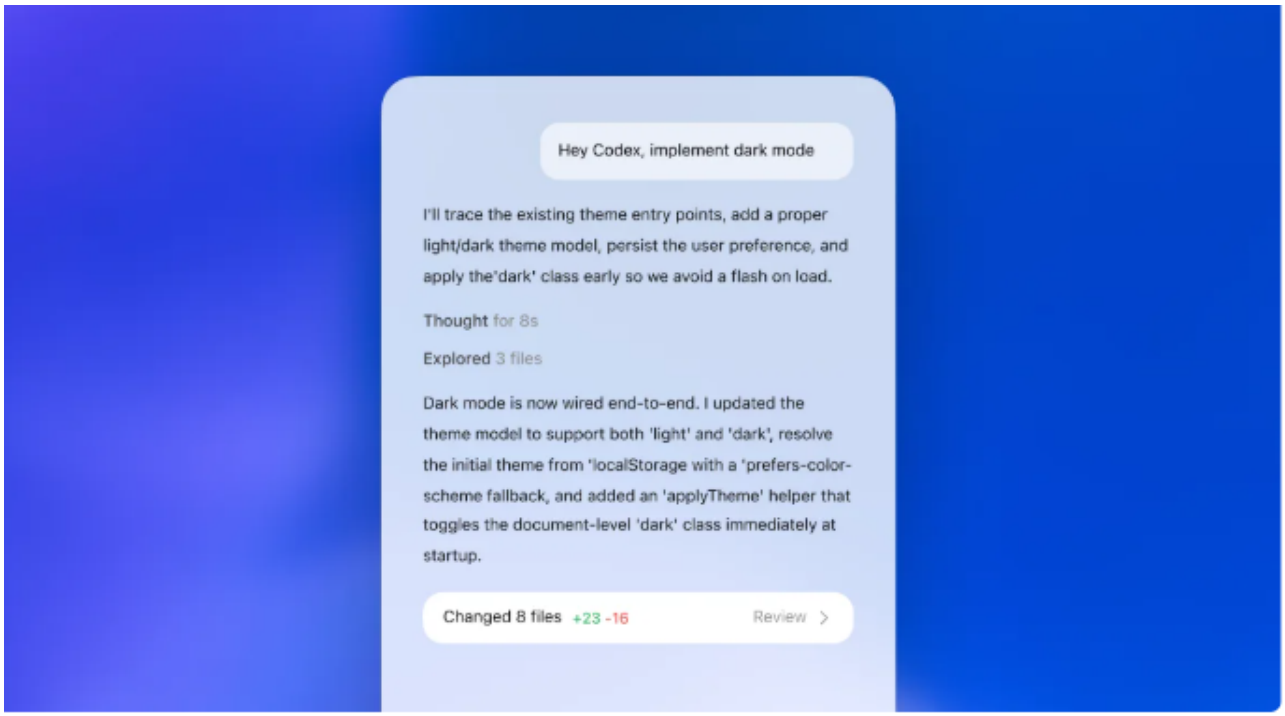
- Codex is framed as a command center for agentic coding, not as a generic chat app. The product language emphasizes end-to-end task execution, parallel work, built-in worktrees, and automations rather than just prompting.
- The information architecture centers on a workspace rail plus a primary work canvas. Images on the official page show persistent project/task navigation, rich status output, commit/review surfaces, and clear transitions between app, editor, and terminal.
- The visual language is softer and more editorial than a typical developer tool. OpenAI uses generous spacing, large headings, calm cards, and lighter interface surfaces to make advanced agent workflows feel approachable rather than overloaded.
- Codex repeatedly surfaces progress and trust cues: changed files, review states, approvals, tests, branch summaries, and explicit continuation points. The UX is designed to answer 'what happened' and 'what can I do next' at every step.
- The product positioning is multi-surface. The app does not stand alone; it is one entry point in a connected workflow that continues in the editor and terminal. That means the UI favors orchestration and oversight as much as composition.

Visual Example: Codex hero interface



The official Codex hero emphasizes a workspace rail, a primary execution canvas, and a composer embedded into the same operational surface.

Visual Example: Codex sidebar and project context

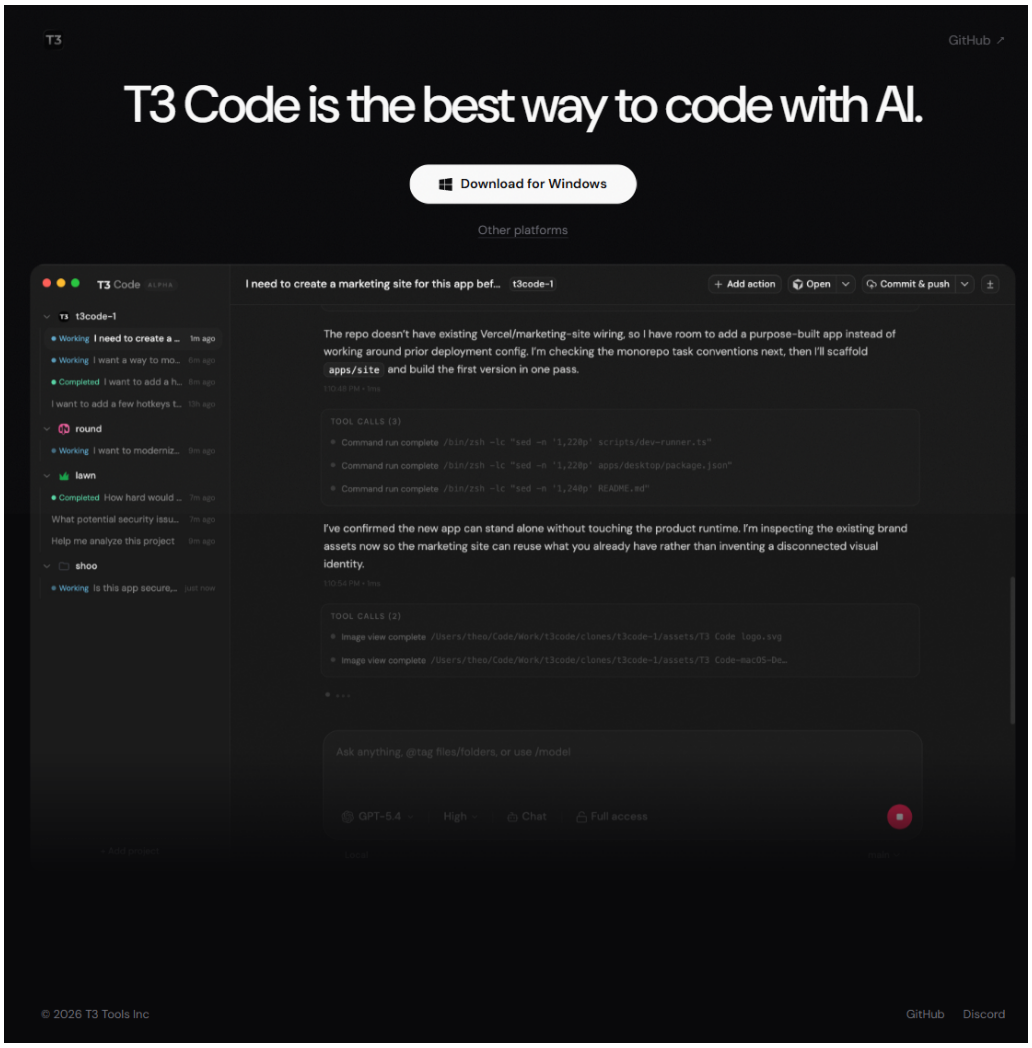


Codex repeatedly highlights project-level context and execution output, reinforcing that the product is about coordinated work rather than isolated prompts.

T3 Code UI/UX Findings

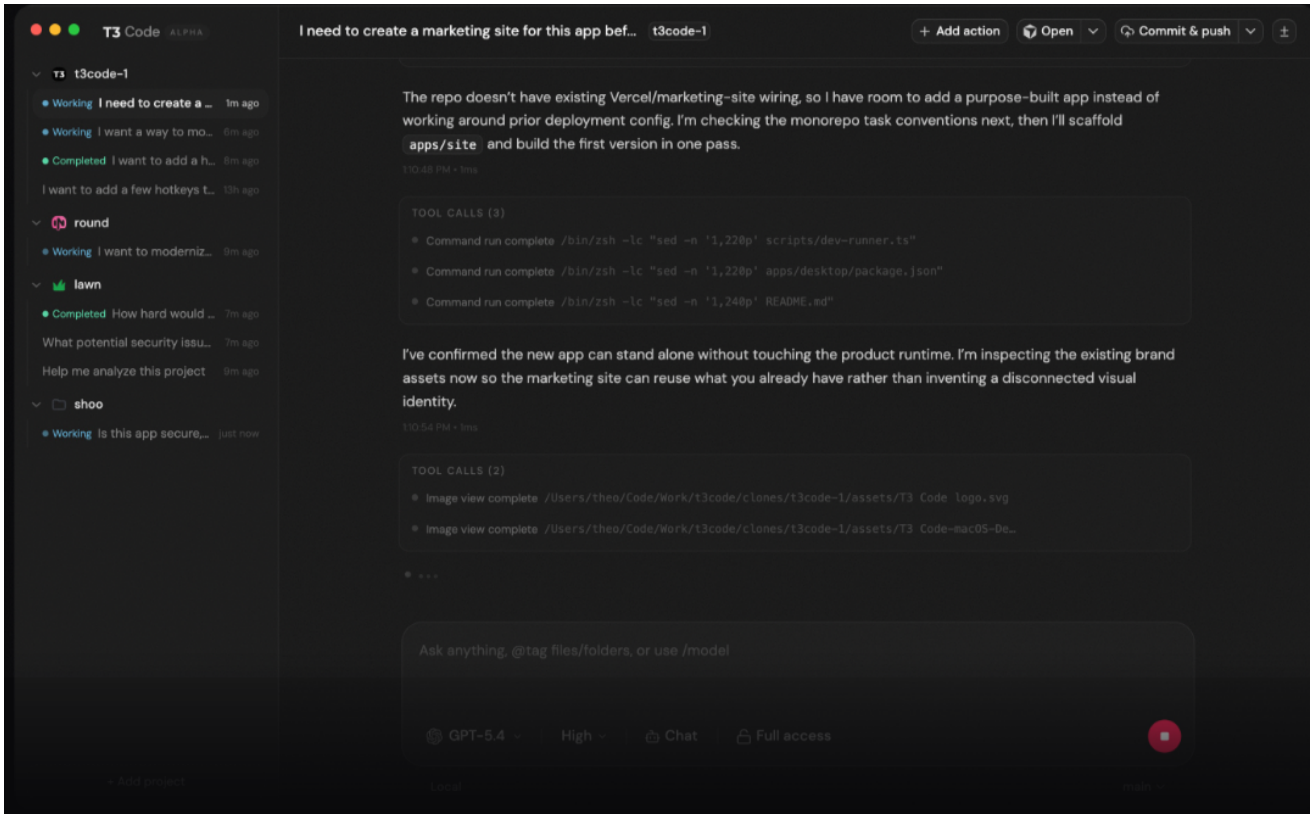
- T3 Code presents itself as a dense, high-focus workstation. The homepage hero shows the entire product value in one frame: a persistent left rail, a compact task list, a transcript-first center pane, top-right action controls, and a bottom composer bar.
- The information architecture is project-centric. The left rail appears to hold projects and active conversations, while the center panel is always about the current thread and its execution log. That makes context switching cheap and keeps navigation predictable.
- The visual language is dark, glassy, and restrained. Most of the UI uses low-chroma neutrals, thin separators, subtle cards, and very sparse accent color. The result feels fast and intentional rather than decorative.
- T3 Code uses density well. The interface fits a lot of state into one screen because every surface is compact: transcript timestamps are small, tool-call blocks stay inline, and the composer exposes model, reasoning, mode, and access settings without opening extra panels.
- The primary interaction loop is always visible. You read the running transcript, understand what the agent is doing, and respond in the same place. That continuity is a major reason the interface feels operational rather than modal.

Visual Example: T3 Code landing page hero



T3 Code frames the entire product value in a single hero: dark shell, dense left rail, transcript-first center, and a bottom composer with inline control state.

Visual Example: T3 Code interface crop



The crop makes the layout logic clearer: projects on the left, an action bar at the top, execution detail in the center, and the composer always visible at the bottom edge.

Applied Design Direction For eCode

The target state for eCode is a dark, compact command-center shell with calmer internal cards and stronger operational trust cues. The shell should borrow T3 Code's density and focus, while inner surfaces should borrow Codex's clarity, progress visibility, and less intimidating task framing.

Recommended Actions

- Strengthen shell hierarchy: treat top bar, sidebar, right panel, status bar, and terminal as one coherent product system.
- Make the thread header operational: provider, model, access mode, and planning mode should read as a compact control strip rather than scattered form controls.
- Improve transcript readability: distinguish user turns, assistant turns, tool activity, waiting states, and approvals more clearly without adding visual noise.
- Turn the composer into the primary action anchor: input, send, stop, model, and context states should feel deliberate and always within reach.
- Unify settings with the main product language: the settings modal should feel like part of the app, not a separate admin panel.
- Use visual trust cues everywhere: file changes, running state, waiting state, errors, and success states must be immediately legible.

What Not To Copy

- Do not copy marketing-only visual treatments that would make the desktop app slower or harder to scan.
- Do not replace eCode's local-first identity with a generic SaaS dashboard posture.
- Do not add extra configuration chrome that breaks the transcript-first workflow.

AI UI Designer Prompt

You are redesigning eCode, a Rust-native desktop coding assistant built with egui. Your job is to analyze the current product deeply, absorb the strongest UI and UX patterns from Codex and T3 Code, and refine the whole program into a coherent, production-grade desktop experience.

Context

- eCode already has a working shell with a top bar, left sidebar, center chat/workspace panel, right plan panel, settings modal, status bar, and optional terminal.
- The product is local-first and desktop-native. Keep that identity.
- The current app recently moved toward a Codex/T3 direction, but it still needs a more complete systems-level refinement pass.
- Codex model selection is now dropdown-based and API key UI is intentionally removed. Do not reintroduce API key entry.

Reference taste to absorb

From Codex

- Treat the app like a command center for real engineering work, not a generic chatbot.
- Preserve strong progress reporting, review/change visibility, and explicit execution states.
- Use calmer cards, clearer sectioning, and more trustworthy operational affordances.
- Make multi-step agent workflows feel manageable, not overwhelming.

From T3 Code

- Preserve a dense, high-focus dark shell with a strong left rail and a transcript-first center pane.
- Keep chrome minimal and useful.
- Make the composer and current thread feel like the natural center of gravity.
- Use compact inline controls and subtle separators instead of bulky forms.

What to redesign

- Top bar
- Sidebar / thread navigation
- Main thread header
- Transcript / tool-call blocks / approvals / waiting states
- Composer
- Right-side plan or context panel
- Settings modal

- Status bar
- Terminal panel
- Empty states and onboarding moments

Design goals

- Make the whole app feel like one intentional product, not a collection of panels.
- Improve information hierarchy and reduce ambiguity about status, next action, and ownership.
- Increase perceived trust and quality.
- Keep the app fast, local, and desktop-native.
- Preserve accessibility, legibility, and responsive behavior for smaller desktop windows.

Non-negotiable constraints

- No API key entry UI.
- Codex model choice must remain a dropdown-based flow.
- Do not turn the app into a browser-clone or marketing mockup.
- Do not add decorative complexity that hurts scan speed.
- Respect the current Rust/egui implementation reality and propose changes that can plausibly be built in this codebase.

Working method

1. Audit each existing surface.
2. Identify the strongest UI/UX patterns worth keeping.
3. Identify the weakest or most incoherent areas.
4. Propose a unified visual direction: layout logic, density model, spacing rhythm, color strategy, card treatment, status chips, and composer behavior.
5. Refine the whole program screen by screen, not just the main chat view.

Deliverables

- A thorough UI/UX analysis of the current eCode shell.
- A refined design direction statement.
- Concrete redesign recommendations for every major surface.
- Suggested component-level changes and interaction rules.
- A prioritized implementation plan for the redesign.

Required final report format

1. Overall design direction

2. What changed in each surface
3. Why each change improves UX
4. Which Codex patterns were borrowed
5. Which T3 Code patterns were borrowed
6. What was intentionally not copied
7. Risks or tradeoffs
8. Next implementation priorities

Be opinionated, specific, and exhaustive. I do not want a shallow 'make it cleaner' pass. I want a designer who understands the product taste, internalizes it, and refines the entire program like a real operating environment for coding agents.

Sources

OpenAI Codex product page: https://openai.com/codex/

OpenAI model reference: https://platform.openai.com/docs/models

OpenAI GPT-5.3-Codex announcement: https://openai.com/index/introducing-gpt-5-3-codex/

T3 Code homepage: https://t3.codes/

T3 Code GitHub repository: https://github.com/pingdotgg/t3code